

Tutorial ZRP Lab

This lab explores Zone Routing Protocol as a hybrid approach: nodes maintain local routes proactively and discover more distant routes by bordercasting. You will build the supplied OMNeT++ model, observe a moving relay and route repair in Qtenv, and analyze destination delivery, one-way delay, hop count, and discovery time. The supplied ZrpMobilityLab folder is self-contained.

1 Prepare the workspace

Use OMNeT++ 6.4.0 or a compatible release with a C++ toolchain. This ZRP teaching model does not require INET: it uses OMNeT++ simple modules, direct packet delivery, a 250 m unit-disk range, and an 80 s run. Keep ZrpNetwork.ned, ZrpNode.h, ZrpNode.cc, omnetpp.ini, and Makefile together. The model was built and run on OMNeT++ 6.4.0; results can differ after edits or version changes.

1. Start the OMNeT++ IDE and choose a workspace. Confirm that its bundled C++ build tools are available. The project folder already includes an OMNeT++ project descriptor and a generated Makefile.

2. Choose File > Import > General > Existing Projects into Workspace. Select the ZrpMobilityLab folder and import the detected project. Copy the folder into the workspace only if you want a separate working copy.

3. In Project Explorer, confirm that ZrpNetwork.ned, ZrpNode.h, ZrpNode.cc, omnetpp.ini, Makefile, and README.md are at the project root. Do not nest the files one level deeper accidentally.

If you instead create a new project, enable C++ support and copy these files into its root. Set that root as the NED source folder and C++ source location. Keep the run configuration's working directory at the folder containing omnetpp.ini.

4. Check Project Properties > OMNeT++ > NED Source Folders and include the project root. The ZRP lab needs no inet project reference. If the IDE does not recognize the existing Makefile, regenerate it with `opp_makemake -f --deep -O out` in an OMNeT++ shell.

5. Build the project with Project > Build Project or run `make` in the OMNeT++ shell from the project root. Resolve C++ or NED errors before starting the simulation.

2 Import an existing OMNeT++ project

The supplied ZrpMobilityLab folder has a .project file, so Existing Projects into Workspace is the shortest route. After import, verify that the project root is a NED source folder and that the C++ target ZrpMobilityLab builds. If a run configuration cannot find the binary, build from the project root and point the launch to that executable. Keep omnetpp.ini beside the executable and NED file.

3 Understand what the files build

File or group	Role in the simulation
ZrpNetwork.ned	Defines ten ZrpNode modules, the canvas legend, and the gold relay trajectory.
omnetpp.ini	Sets node positions, range, zoneRadius, run length, seed, and scalar/vector recording.
ZrpNode.h, ZrpNode.cc	Implement zone updates, bordercast discovery, source routes, relay movement, message labels, and metrics.
Makefile	Builds the self-contained ZrpMobilityLab C++ executable with OMNeT++ tools.
README.md, .project	Explain the model and let the IDE import the folder as a project.

The ten-node geometry matches the AODV mobility lab. Initially, a central candidate route is host[0] -> host[1] -> host[2] -> host[3] -> host[9] (four links). The upper bypass can use host[0] -> host[4] -> host[5] -> host[6] -> host[3] -> host[9] (five links). A two-hop zone is maintained from periodic link-state messages. Beyond the zone, route queries travel toward peripheral nodes. These are possible routes, not fixed forwarding instructions.

4 Run ZRP

- 1. Open omnetpp.ini and run the General configuration. Right-click the ini file and choose Run As > OMNeT++ Simulation, or create a simulation launch. Select Qtenv to view node movement and packet animation.**
- 2. In Run Configurations, use the folder containing omnetpp.ini as the working directory. Select the built ZrpMobilityLab executable and the ZrpNetwork NED path. Run to the 80 s limit; pause around the times in the observation table.**
- 3. Watch host[0] (SOURCE), host[9] (DESTINATION), and yellow host[2] (MOVING RELAY). The message names in the animation or event log distinguish ZONE UPDATE, BORDERCAST, ROUTE REPLY, PING, and ROUTE ERROR.**

The source starts sending pings at 4 s and attempts one every 0.6 s. Each node sends a zone update every 1 s. The ini file sets seed-set = 41, a two-hop zone radius, and 250 m communication range. Repeating the same configuration with the same seed is reproducible, but it is not an independent trial.

What the custom code measures

ZrpNode.cc implements link-state updates limited to the local zone, bordercast route requests, replies, source-routed data, and errors after broken next hops. The source counts attempted pings and reads the destination received count at finish(). It records delivery ratio, discovery delay, bordercast branches, protocol packet totals, and route errors. The destination records one-way delay and hop count. Named vectors preserve the time history of delay, hops, discovery delay, zone size, active route hops, packet kind, control/data traffic, ping attempts and deliveries, and route errors.

Parameter guide. sim-time-limit controls how long the run observes the topology; 80 s includes the relay departure, stationary interval, and return. seed-set controls repeatability (the supplied topology and movement are mostly deterministic). range is the unit-disk communication radius; lowering it breaks links sooner, while increasing it can hide route repairs. zoneRadius is the proactive radius in hops; a larger value reduces bordercast distance but increases zone-update overhead. The node x and y values define geometry. Ping start time (4 s), ping interval (0.6 s), zone-update period (1 s), and the relay timetable are code constants in this teaching model, so changing them requires editing ZrpNode.cc and rebuilding.

5 Observe the route change

Simulation time	What to inspect
0-18 s	Local zone updates begin; the first route query and reply precede early ping deliveries on a central path.
18-24 s	The yellow relay moves from (520,325) to (520,500). Watch a forwarding break and fresh bordercast discovery.
24-36 s	The relay stays at (520,500). Check whether the upper bypass carries data and whether hop-count samples change.
36-42 s	The relay moves back toward (520,325). Observe route expiry, rediscovery, and resumed central connectivity.
42 s onward	Inspect the later selected path, then finish the run so summary scalars and vectors are written.

The exact break occurs when a candidate next hop exceeds the configured range; it need not coincide with the first instant of movement. A zone update refreshes local topology, while a bordercast explores beyond the zone. Route replies establish a path and a route error reports a broken forwarding hop. Read message names or the event log to distinguish control traffic from PING packets. Animation speed changes only viewing time.

6 Find and analyze results

1. Let the run reach 80 s, refresh Project Explorer, and open results. The .sca file contains summary scalars, the .vec file contains the enabled time-series vectors, and the .vci file indexes those vectors. The event log is recorded separately.
2. In Browse Data, filter on ZrpNetwork.host[0] for sent, delivered, lost, delivery ratio, discovery delay, bordercast branches, control/data totals, and route errors. Filter on ZrpNetwork.host[9] for mean one-way delay and mean hops. Check vector names before plotting so you do not confuse a packet-kind trace with a delay trace.
3. Plot host[9].hopCount and host[9].oneWayDelay against simulation time. Plot host[0].routeDiscoveryDelay, zoneSize, and activeRouteHops to see topology knowledge and selected paths. PingAttempt and pingDelivered show offered load and successful delivery; packetKindTransmitted and the control/data vectors show overhead.
4. Compare an initial route, the interval with the relay away, and the later recovery window. Report both a path from the event log and a destination metric. For a protocol comparison with AODV or DSDV, first align traffic start, run length, range, offered load, recording settings, and multiple seeds.

Recorded result	How to interpret it
ping packets sent by source	Number of scheduled ping attempts at host[0], including attempts while discovering a route.
ping packets received by destination	Data packets that actually reached host[9]; no echo replies are modeled.
ping packets lost before destination	Attempt count minus destination deliveries.
packet delivery ratio at destination (%)	100 x destination deliveries / source attempts; zero attempts report 0.
average one-way delay at destination (s)	Mean creation-to-arrival interval for delivered PING packets.
average hops to destination	Mean number of links in routes that delivered PING packets.

Recorded result	How to interpret it
route discovery delay mean (s)	Mean time from starting a query to receiving its first current route reply.
zoneSize, activeRouteHops, packet/control/data/ping/routeError vectors	Time histories of zone knowledge, route length, offered load, protocol overhead, deliveries, and broken-route events.

The source counts each ping attempt even if discovery is pending, so some attempts have no data packet on the route. Delivery ratio is destination-received divided by attempts, not an ICMP reply rate. A single demonstration run illustrates behavior but is not a statistical comparison across protocols. Changing range, zoneRadius, or traffic timing changes both delivery and control overhead, so record the parameter values beside every result.

7 Student analysis questions

- Identify the first discovered path and its hop count. Use the event log to name the query, reply, and forwarding steps that precede the first delivered ping.
- Locate the relay departure and the next route discovery in the event log. When does the central route become unusable, and which path carries later pings?
- Compare a local ZONE UPDATE with an inter-zone BORDERCAST. How does increasing zoneRadius change proactive overhead and query distance?
- Repeat the run with a smaller range or a larger zoneRadius. Predict the direction of change for delivery ratio, discovery delay, and control/data packet counts, then compare with the vectors.
- Report sent, destination-received, lost, destination PDR, mean one-way delay, mean hops, discovery delay, bordercast count, and the recording parameters. State the 80 s run length and 4 s traffic start beside the values.

8 Troubleshooting and limits

- Unknown ZrpNode NED type: confirm ZrpNetwork.ned is in a NED source folder and the executable was rebuilt after changing ZrpNode.cc or ZrpNode.h.
- Executable or custom module not found: build with the supplied Makefile in an OMNeT++ shell, then select the resulting ZrpMobilityLab executable in the run configuration.
- No packets or only a blank network: run the General configuration in omnetpp.ini with ZrpNetwork as the network, and confirm the working directory contains the ini file.
- No summary scalars or vectors: let the 80 s simulation complete, keep scalar-recording and vector-recording enabled, and refresh the results folder afterward.
- No colored per-hop arrows: this self-contained model uses direct sends. Qtenv animates packets and labels, while the colored legend names message roles; it does not install INET canvas visualizers.
- This is an instructional hybrid-routing model, not a full ZRP or wireless stack implementation. It omits contention, interference, security, and IP forwarding. Treat quantitative comparisons with INET labs as illustrative unless the models are aligned.

Reference documentation

OMNeT++ User Guide: <https://doc.omnetpp.org/omnetpp/UserGuide.pdf>

Simulation source: ZrpMobilityLab/ZrpNode.cc and ZrpNetwork.ned